

Kungliga Tekniska Högskolan

SCHOOL OF ELECTRICAL ENGINEERING
AND COMPUTER SCIENCE

DD2367

QUANTUM COMPUTING FOR COMPUTER SCIENTISTS

FINAL PROJECT

Shor's Factorization Algorithm

Florian Jerome Immig
immig@kth.se

Georgios Vidalakis
gvid@kth.se

Roger Vendrell Colet
rogervc@kth.se

December, 2024

Contents

1	The problem: integer factorization	1
2	Method	1
3	Order-finding algorithm	1
3.1	Quantum Fourier transform	1
3.2	Adder gate	2
3.3	Modular adder gate	3
3.4	Controlled multiplier gate	3
3.5	Controlled multi-qubit swap gate	4
3.6	Controlled U_a gate	5
3.7	Quantum circuit with $4n + 2$ qubits	5
3.8	Quantum circuit with $2n + 3$ qubits: the one-qubit trick	7
4	Results	8
5	Discussion	9

1 The problem: integer factorization

Integer factorization refers to the problem of factoring an integer number into its prime factors. As of today, there is no known algorithm that solves the problem in polynomial time on a classical computer. The problem of integer factorization is of great importance as important cryptographic algorithms rely on it being hard to solve. Among these is the Rivest-Samir-Adleman (RSA) cryptosystem which is widely used for encryption today. Shor's Algorithm, first proposed by Peter Shor in [3], is a way of factoring integers in polynomial time with the help of a quantum algorithm that can be executed on a quantum computer. Therefore, using Shor's algorithm could allow to break the RSA crypto-scheme if one has a quantum computer with a sufficient amount of qubits.

As one of the fundamental constraints of today's quantum computers is their amount of qubits, researchers try to reduce the number of qubits needed to implement Shor's algorithm. Using today's quantum computers one can not use Shor's algorithm to run on problems of practical relevance.

2 Method

In this project we implement Shor's algorithm using Python and Qiskit, by following a paper by Beauregard [2] who suggests a circuit for implementing Shor's algorithm (i.e. the order-finding algorithm) that uses $2n + 3$ qubits. The whole algorithm including the non-quantum steps is the following, where N is the number to be factored:

1. In the case that N is even, return 2 as a factor.
2. Check whether $N = p^q$ for $p \geq 1, q \geq 2$, if it is return p as a factor.
3. Choose a random a , such that $1 < a \leq N - 1$. Then calculate $\gcd(a, N)$ and if $\gcd(a, N) > 1$ (i.e. a and N are not coprime) we return the common factor $\gcd(a, N)$.
4. Use the order-finding algorithms in order to find the order r of $a \pmod N$.
5. In the case of r being odd or r being even but $a^{r/2} \equiv -1 \pmod N$ start over from step 3. Else, we calculate $\gcd(a^{r/2} - 1, N)$ and $\gcd(a^{r/2} + 1, N)$. For both numbers, check whether they are a non-trivial factor of N and if so, return this factor.

In order to find the prime factorization, we can repeatedly call the above-described algorithm. Steps 1, 2, 3 and 5 work in a completely classical way. Step 4 however makes use of quantum technology, as quantum computers allow us to find the order r of $a \pmod N$ in polynomial time. The order r refers to the smallest positive integer that satisfies the equation $a^r \equiv 1 \pmod N$. In the following we cover the details about this variation of the order-finding algorithm including the necessary gates and steps to perform.

3 Order-finding algorithm

Our implementation of the order-finding algorithm is based on the design of quantum gates that are built sequentially, meaning each gates uses the previous ones. This process allows for thorough testing of each gate before using it to build further components of the circuit.

3.1 Quantum Fourier transform

The first gate we need to implement is the quantum Fourier transform (QFT), the quantum analog of the discrete Fourier transform. Its goal is to encode integers using phase shifts instead of whole states. As we can see in Figure 1, state $|0\rangle$ encoded with whole-state encoding gets translated to $|\phi(0)\rangle$, in

what we call the Fourier space (ϕ -space for short). This new encoding relies on the usage of all qubits, where their relative phase shifts determines the integer to be encoded. In this case, no phase shifts encode state $|\phi(0)\rangle$, whereas state $|\phi(1)\rangle$ would be represented by consecutive phase shifts of π radians each. State $|\phi(2)\rangle$ would be represented by phase shifts of $\frac{\pi}{2}$ radians. Further states are encoded by continuing to increase the resolution of the phase shifts.

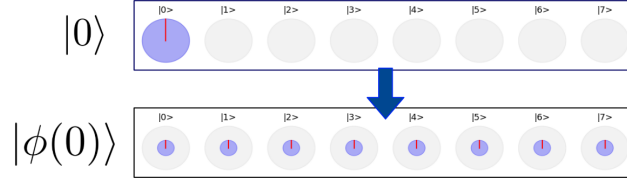


Figure 1: Effect of applying the Quantum Fourier Transform to a register that holds state $|0\rangle$ in regular encoding.

The implementation of the QFT gate only relies on the use of Hadamard gates and controlled Phase gates. They are used following a recursive design, where at each step we apply a Hadamard on the most-significant qubit, and then apply a set of rotations, each controlled by one of the remaining qubits. Once this is done, we discard the most-significant qubit and repeat the same process again, now with $n - 1$ qubits. This process is applied a total of n times, as we can see in Figure 2:

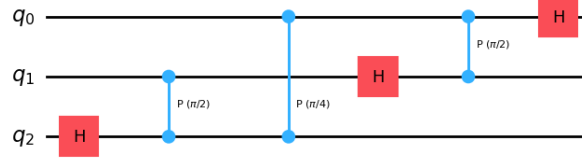


Figure 2: Implementation of the Quantum Fourier Transform gate. Example on 3 qubits.

In order to test the gate we have assessed its capacity to encode any integer in the ϕ -space. At the same time, we have checked whether its inverse gate (the inverse QFT) is able to retrieve the original encoding from the frequency information in the ϕ -space.

3.2 Adder gate

Now that we can already encode integers in the ϕ -space, we can start implementing simple arithmetic operations that will be used in more complex gates afterwards. The adder gate implements addition (and its inverse, subtraction) in the Fourier space.

$$\begin{aligned}
 |\Phi(b)\rangle &= \begin{bmatrix} \Phi \\ A \\ D \\ D \\ (a) \end{bmatrix} = |\Phi(a+b)\rangle & \quad |\Phi(b)\rangle &= \begin{bmatrix} \Phi \\ A \\ D \\ D \\ (a) \end{bmatrix} \begin{cases} |b-a\rangle & \text{if } b \geq a \\ |2^{n+1}-(a-b)\rangle & \text{if } b < a \end{cases} \\
 \text{(a) Effect of addition.} & & \text{(b) Effect of inverse addition.}
 \end{aligned}$$

Figure 3: A compact representation of the adder gate, by [2].

One can easily intuit that addition in the ϕ -space will only involve specific rotations on each of the qubits. As we can see in Figure 4, adding the integer 5 to a 3-qubit register that is already in the ϕ -space yields a total of 3 rotations, which can be controlled by none, one or two qubits if needed.

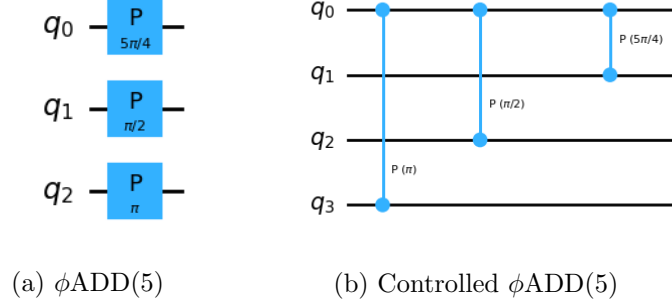


Figure 4: Examples of our implementation of the adder gate on 3-qubit registers.

Testing the adder gate involved doing thorough testing on the conditions stated in Figure 3, where subtraction is a bit more tricky. For all of the cases, we tested the adder and inverse adder gates the following way: first, we encoded a register in the ϕ -space using QFT, then we applied addition/subtraction and, finally, we used the inverse QFT to check whether we could retrieve the correct result of the operation.

3.3 Modular adder gate

$$|\Phi(b)\rangle = \begin{array}{c} \Phi \\ A \\ D \\ D \\ (a) \\ \text{mod} \\ N \end{array} = |\Phi((a+b) \bmod N)\rangle$$

Figure 5: A compact representation of the modular adder gate by [2].

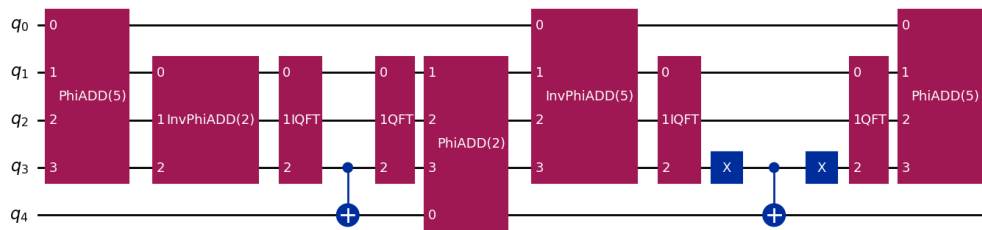


Figure 6: Example of our implementation of the modular adder gate.

3.4 Controlled multiplier gate

The controlled multiplier gate implements a controlled modular multiplication and addition, as shown in Figure 7. Specifically, given two quantum numbers x and b , regularly encoded in dedicated quantum

registers, as well as a controlled qubit, c , this quantum gate modifies the state of the b register (if the control qubit is 1) so that it becomes $(b + a \cdot x) \bmod N$, where a and N are known, classical numbers. Note that if b is 0 and the control qubit is 1, then the content of the b register simply becomes $(a \cdot x) \bmod N$.

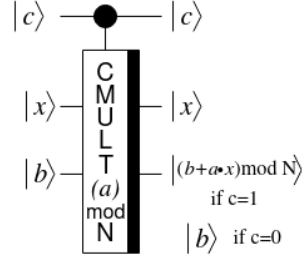


Figure 7: A compact representation of the controlled multiplier gate by [2].

The implementation of the controlled multiplier gate (see Figure 8) involves the utilization of previously implemented gates. Firstly, a QFT gate is applied on the b register to translate b to the ϕ -space. Then a series of doubly-controlled modular adder gates is applied, the i -th of which is control both by the control qubit and by the i -th qubit of the x register. Finally, an IQFT gate is applied on the b register to translate the result back to the regular space.

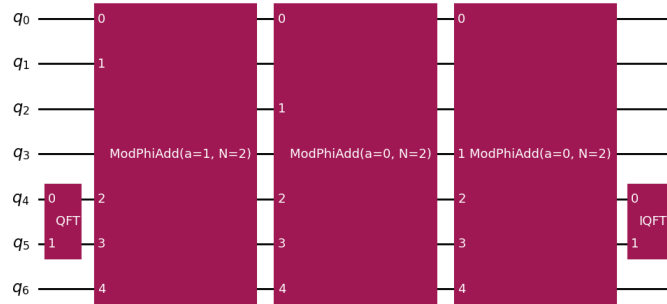


Figure 8: Example of our implementation of the controlled multiplier gate.

To test this gate, we executed multiple automated test in a quantum state-vector simulator. For each test the control qubit was initialized to either 0 or 1, while the x and b registers were initialized to random integers. After each initialization, the gate was applied and the resulting state vector was used to verify that the result was the expected one.

3.5 Controlled multi-qubit swap gate

This gate is the simplest of the series of gates that we have implemented. It allows us to change the contents of two quantum registers, both having the same number of qubits. This operation can be controlled by an additional control qubit.

As we can see in Figure 10, the controlled multi-qubit swap is basically a chain of controlled swaps. Each of the swaps affects qubits that share the same level of significance in both registers. This means, for example, that the least-significant qubit from register 1 is swapped with the least-significant qubit from register 2, and so on. Needless to say, the testing of this gate is straight-forward.

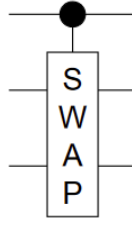


Figure 9: A compact representation of the controlled multi-qubit swap gate, by [2].

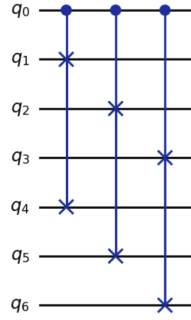
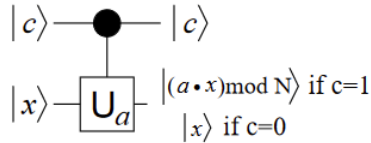


Figure 10: Example of our implementation of the controlled multi-qubit swap gate.

3.6 Controlled U_a gate

At this point, we are able to take $|x\rangle|b\rangle$ to $|x\rangle|b + (a \cdot x) \bmod N\rangle$ by using the controlled multiplier gate. However, what we need is one last gate that takes $|x\rangle$ to $|a \cdot x \bmod N\rangle$. We can achieve this by implementing the controlled U_a gate.

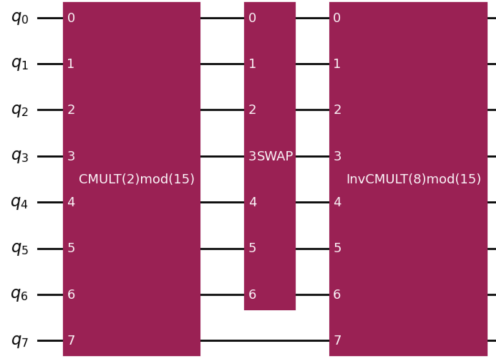
Figure 11: A compact representation of the U_a gate, by [2].

As we can see in Figure 12, its implementation consists in a $CMULT(a)MOD(N)$ operation, followed by a controlled swap and, finally, an $inverseCMULT(a^{-1}MOD(N))$ operation. This process takes registers $|x\rangle|0\rangle$ to $|a \cdot x \bmod N\rangle|0\rangle$, as proved in Figure 13.

Testing of the controlled U_a gate involved ensuring correct behaviour on both registers for different values of x , a and N , also testing different register sizes.

3.7 Quantum circuit with $4n + 2$ qubits

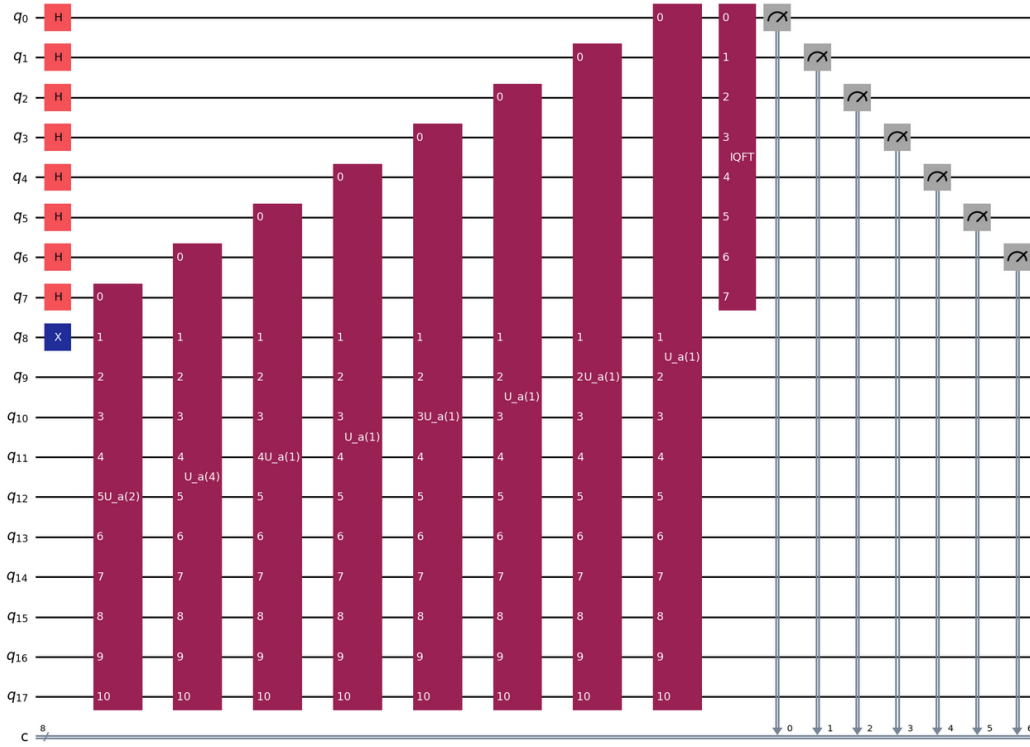
To compute the order of the number of interest, we use a quantum circuit with $4n + 2$ qubits (see Figure 14). The first $2n$ qubits are initialized in a superposition of all states by applying a Hadamard gate to each of them. Then, they are used to control a sequence of U_a gates; the i -th of them (indexing from one) controls a $U_a(a^i \bmod N)$ gate. This controlling makes the phases of the first $2n$ qubits change, so when the IQFT gate is applied on them afterwards, the amplitude of some states becomes

Figure 12: Example of our implementation of the controlled U_a gate.

$$\begin{aligned}
 |x\rangle |0\rangle &\rightarrow |x\rangle |(a \cdot x) \bmod N\rangle \\
 &\rightarrow |(a \cdot x) \bmod N\rangle |x\rangle \\
 &\rightarrow |(a \cdot x) \bmod N\rangle |(x - a^{-1} \cdot a \cdot x) \bmod N\rangle \\
 &= |(a \cdot x) \bmod N\rangle |0\rangle
 \end{aligned}$$

Figure 13: Transformation of the 2 used registers under the controlled U_a gate arithmetic.

larger. Finally, after the IQFT, the first $2n$ qubits are measured and the number of states with a spike is equal to the period of a , as illustrated in Section 4.

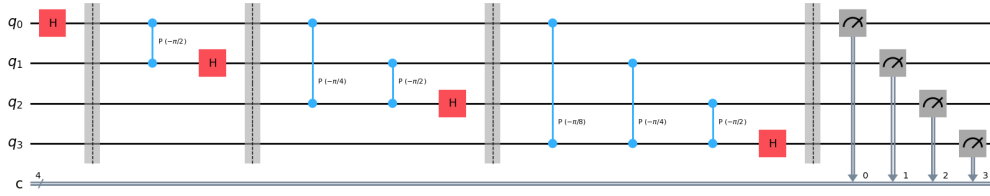
Figure 14: Example of our implementation of the order-finding circuit with $4n + 2$ qubits.

3.8 Quantum circuit with $2n + 3$ qubits: the one-qubit trick

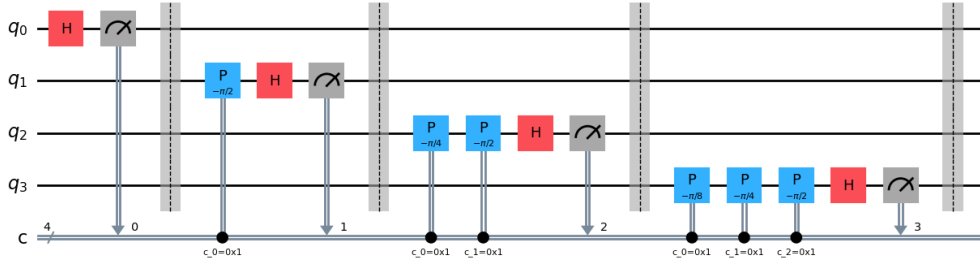
The number of qubits can be further reduced by applying the one-qubit trick. This trick allows merging the first $2n$ qubits of the $4n + 2$ circuit and getting a circuit with the same distribution over measurements as the original one. The one-qubit trick relies on two facts.

Firstly, one can consider a circuit consisting of $2n$ qubits on which an IQFT is applied before they get measured, as shown in Sub-Figure 15a. The deferred measurement principle [1], implies that an equivalent circuit can be obtained if each measurement is moved before the controlled-phase gates of the corresponding qubit, while these controlled-phase gates become classically controlled. The resulting circuit is shown in Sub-Figure 15b.

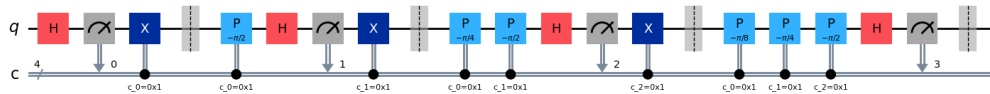
Secondly, one can observe that the measurements will not be affected if the $2n$ qubits of the circuit shown in Sub-Figure 15b merge, as shown in Sub-Figure 15c.



(a) The IQFT and measurement circuit part initially.



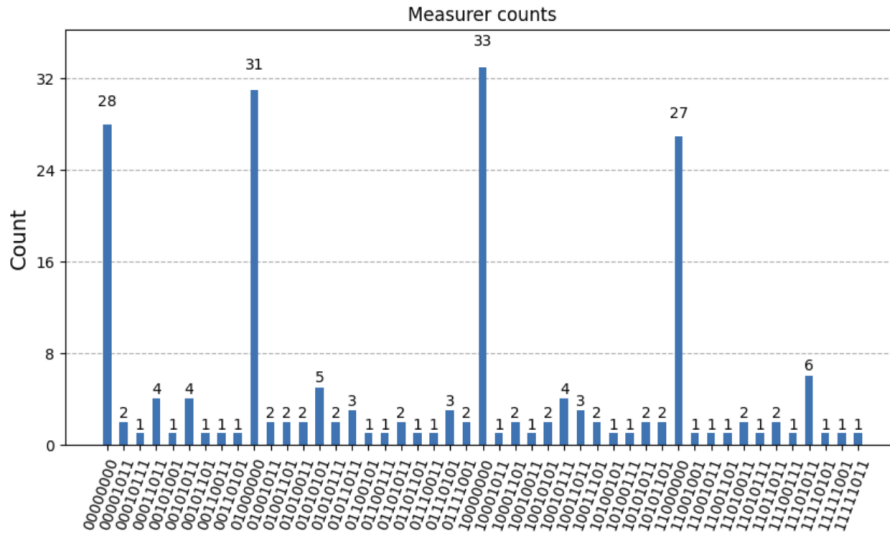
(b) The IQFT and measurement circuit part after using the deferred measurement principle.



(c) The IQFT and measurement circuit part after merging the qubits.

Figure 15: Reducing the number of qubits by using the one-qubit trick.

By applying the one-qubit trick on the quantum circuit with $4n + 2$ qubits, while accounting for the U_a gates, an equivalent quantum circuit with just $2n + 3$ qubits can be obtained. $2n + 3$ is the minimum number of qubits that is sufficient for implementing a general order-finding quantum circuit, based on the current public scientific knowledge [2].

Figure 16: Factoring 15 using $a = 2$

4 Results

For the evaluation of our circuit we conducted several experiments in a quantum simulator environment. These experiments aimed to factor the numbers 15, 21 and 33 using the circuit described in Section 3.7. For simplicity, we used a fixed a here.

We start by factoring 15 using $a = 2$ where we obtain the counts shown in Figure 16. In this example we use 200 shots which takes less than 1 second to execute on our quantum simulator. Looking at the counts we can see $r = 4$ spikes, and by calculating the first candidate

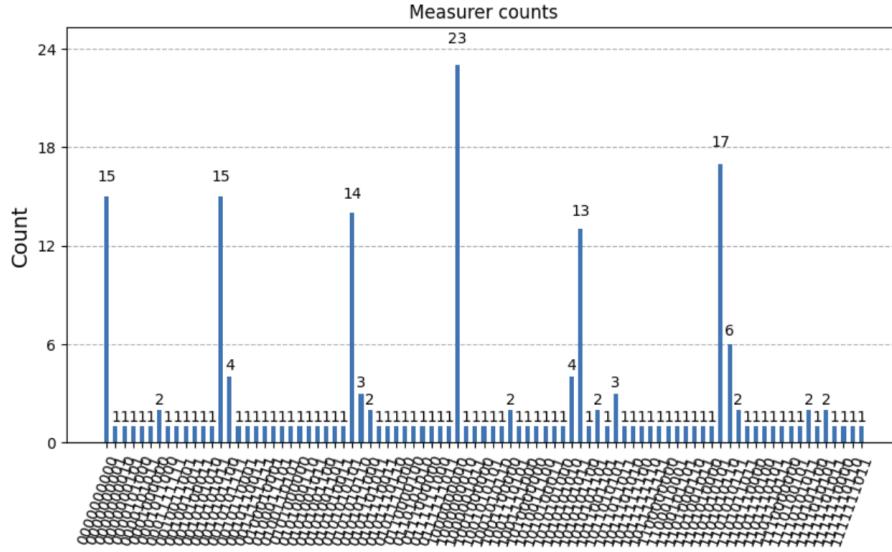
$$\begin{aligned}
 & \gcd(N, a^{\frac{r}{2}} - 1) \\
 &= \gcd(15, 2^{\frac{4}{2}} - 1) \\
 &= \gcd(15, 3) \\
 &= 3 \neq 1
 \end{aligned}$$

and then similarly the second candidate

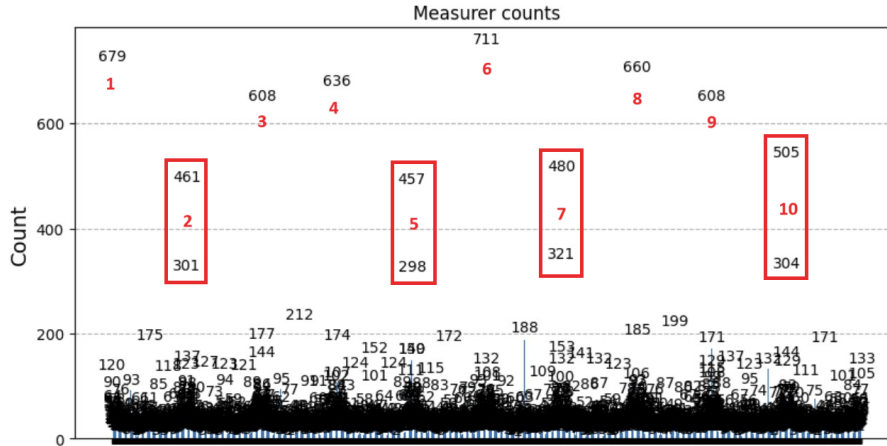
$$\begin{aligned}
 & \gcd(N, a^{\frac{r}{2}} + 1) \\
 &= \gcd(15, 2^{\frac{4}{2}} + 1) \\
 &= \gcd(15, 5) \\
 &= 5 \neq 1.
 \end{aligned}$$

Then we check that both candidates are actually divisors of 15 which is the case here. Following this we check that $3 \cdot 5 = 15$ to confirm that we have found the factorization of 15.

Similarly, we factor $N = 21$ using $a = 2$ using 200 shots, which takes approximately 10 seconds to execute. We obtain the counts seen in Figure 17. Using the observed number of counts $r = 6$ we obtain the candidates 3 and 7 (using a similar calculation as in the previous example) which we can easily confirm to be divisors of 21. Finally, checking that $7 \cdot 3 = 21$ confirms that we have successfully found the factorization of 21.

Figure 17: Factoring 21 using $a = 2$

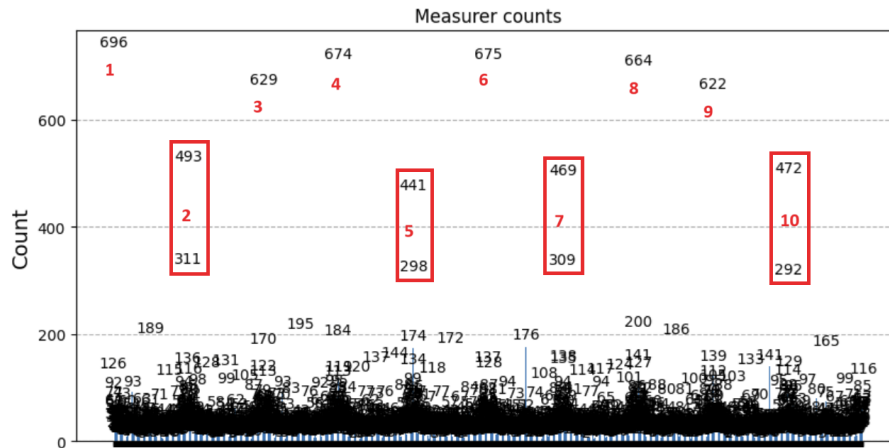
Next we try to factor $N = 33$. In a first attempt, we again use $a = 2$ and obtain the counts shown in Figure 18. The red boxes indicate peaks in the counts that seem to lie between two integers but can be added to obtain a single spike. In our implementation we fix this problem by using bins of size two to add nearby counts and obtain the correct number of peaks, which is $r = 10$ in this case. However, we cannot use these counts as $a^{\frac{r}{2}} = 2^{\frac{10}{2}} = 32 = -1 \pmod{33}$. As described in the algorithm we need to use a different a .

Figure 18: Attempt to Factor 33 using $a = 2$

Hence, we try factoring $N = 33$ again, now using $a = 5$. As seen in Figure 19 we find $r = 10$ peaks in the counts. This time however, $a^{\frac{r}{2}} = 5^5 = 3125 = 23 \pmod{33} \neq -1 \pmod{33}$. After obtaining the candidates 3 and 11 we can then confirm that $3 \cdot 11 = 33$ and we have found the factorization of N .

5 Discussion

As we have seen, we have a working implementation of Shor's algorithm. However there are a number of problems that prevent one from using this circuit (or a variation of it) for factorizing large integers.

Figure 19: Factoring 33 using $a = 5$

First of all, factoring integers still requires a large number of qubits. Therefore, especially factoring integers that are used for cryptography is not possible with today's quantum computers. To be able to factor these kinds of numbers one would require many more qubits. To help with this one can reduce the number of qubits the circuit needs as well as developing better quantum computers (with more qubits).

The largest factor for the infeasibility of using Shor's algorithm however is that one would need lots of additional qubits for quantum error correction as the circuit depth gets deeper for factoring bigger numbers.

References

- [1] Deferred measurement principle - Wikipedia — en.wikipedia.org. https://en.wikipedia.org/wiki/Deferred_measurement_principle. [Accessed 09-12-2024].
- [2] S. Beauregard. Circuit for shor's algorithm using $2n+3$ qubits. *Quantum Info. Comput.*, 3(2):175–185, Mar. 2003.
- [3] P. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, 1994.